

SEG3502 – Lab 4

Angular – Routage

Ce laboratoire comprend deux parties. La première partie fournit une brève introduction à [Cucumber](#), un outil de test pour les tests d'acceptation BDD automatisés. Le second objectif de ce laboratoire est de démontrer le routage d'Angular. Nous allons construire un exemple d'application qui permet de naviguer entre les différentes «pages» d'une application.

I. Cucumber

[Cucumber](#), un outil de test pour les tests d'acceptation BDD automatisés. Les cas de tests sont rédigés dans un langage naturel compréhensible par les clients (Gherkin). L'outil mappe ces cas de test au code d'exécution à l'aide de code de pontage écrit par les développeurs.

Nous nous concentrons sur [Cucumber-JVM](#) qui est l'implémentation des langages JVM. Il convient cependant de noter que Cucumber est disponible pour une variété d'autres langages et plates-formes de programmation.

L'exemple discuté ici est disponible à <https://github.com/stephansome/cucumberExampleDate.git>.

Plugins

L'utilisation de plugins est recommandée pour un meilleur support de l'édition des features Cucumber avec Gherkins et leur exécution. Des plugins sont disponibles pour les IDE populaires tels que [Visual Studio Code](#), [IntelliJ IDEA](#) ou [Eclipse](#). Vous pouvez installer ces plugins via le Plugins Marketplace des IDE respectifs.

Configuration Gradle

Cucumber peut être installé dans un projet avec Gradle. Pour exécuter les tests Cucumber, les dépendances suivantes doivent être présentes :

```
testImplementation("io.cucumber:cucumber-java8:VERSION")
```

```
testImplementation("io.cucumber:cucumber-junit-platform-engine:VERSION")
```

VERSION désigne la version de la dépendance

La dépendance "io.cucumber:cucumber-java" pourrait être utilisée alternativement à "io.cucumber:cucumber-java8".

Une dépendance supplémentaire pour les projets Spring est "io.cucumber:cucumber-spring".

Application NextDate

L'application testée est une implémentation Kotlin d'une application qui détermine la date du jour suivant 'une date donnée. L'application est une implémentation des exigences fonctionnelles suivantes :

L'application doit prendre en entrée une date au format ANNÉE/MOIS/JOUR où ANNÉE est le numéro de l'année, MOIS le nom du mois et JOUR est le numéro du jour, et retourner la date suivante dans le même format.

Si l'entrée n'est pas conforme aux spécifications, l'application doit en informer l'utilisateur.

Les tests ciblent la fonction getNextDate (dans src/main/kotlin/Main.kt). La fonction prend un string en entrée et retourne un string. La fonction getNextDate utilise le type de données MyDate (dans src/main/kotlin/date/MyDate.kt).

Features

Les features sont spécifiées dans le dossier src/test/resources/seg3x02.date. La feature « Next date for valid date »

(src/test/resources/seg3x02.date/is_next_date_right_for_valid_date.feature) comprend cinq scénarios testant différentes classes de dates.

```
1  Feature: Next date for valid date
2      Next date should respond a valid date for a valid input
3
4      Scenario: The day after 1980/December/15 is 1980/December/16
5          Given the valid date "1980/December/15"
6          When I ask for the next date
7          Then the response should be "1980/December/16"
8      Scenario: The day after 2000/February/28 is 2000/February/29
9          Given the valid date "2000/February/28"
10         When I ask for the next date
11         Then the response should be "2000/February/29"
12     Scenario: The day after 1600/February/29 is 1600/March/1
13         Given the valid date "1600/February/29"
14         When I ask for the next date
15         Then the response should be "1600/March/1"
16     Scenario: The day after 1977/December/31 is 1978/January/1
17         Given the valid date "1977/December/31"
18         When I ask for the next date
19         Then the response should be "1978/January/1"
20     Scenario: The day after 25/February/28 is 25/March/1
21         Given the valid date "25/February/28"
22         When I ask for the next date
23         Then the response should be "25/March/1"
```

Ces scénarios peuvent être regroupés en un seul `scenario outline` ayant des valeurs de test spécifiques fournies dans un tableau, tel que dans
(`src/test/resources/seg3x02.date/is_next_date_right_for_valid_date_outline.feature`).

```
1  Feature: Next date for valid date
2      Next date should respond a valid date for a valid input
3
4  Scenario Outline: The day after a valid date is a valid date
5      Given the valid date <date>
6      When I ask for the next date
7      Then the response should be <nextDate>
8
9  Examples:
10     | date                | nextDate                |
11     | "1980/December/15" | "1980/December/16"    |
12     | "2000/February/28" | "2000/February/29"    |
13     | "1600/February/29" | "1600/March/1"        |
14     | "1977/December/31" | "1978/January/1"      |
15     | "25/February/28"   | "25/March/1"          |
```

Steps Definition

Cucumber exécute chaque scénario de feature en faisant correspondre ses étapes avec du code spécifié dans un fichier de définition d'étapes (step definition file). Un tel fichier de définition d'étapes (`StepsDefinition.kt`) est fourni dans le package `cucumber` du dossier source `src/test/kotlin`.

```

1    package cucumber
2
3    import getNextDate
4    import io.cucumber.java8.En
5
6    class StepsDefinition: En {
7        private var date: String = ""
8        private var nextDate: String = ""
9
10       init {
11           Given("the valid date {string}") { date: String ->
12               this.date = date
13           }
14           Given("the invalid date {string}") { date: String ->
15               this.date = date
16           }
17           When("I ask for the next date") {
18               this.nextDate = getNextDate(date)
19           }
20           Then("the response should be {string}") { expectedDate: String ->
21               assert(nextDate == expectedDate)
22           }
23       }
24   }

```

Nous utilisons ici la *syntaxe lambda* pour les définitions d'étapes (une syntaxe de fonction régulière pourrait également être utilisée). Notez qu'en l'absence de définition d'étape correspondante pour une étape de fonctionnalité, Cucumber présente un message d'erreur avec un template pour cette étape manquante. Cela peut être utilisé comme point de départ pour l'implémentation des définitions d'étapes.

Test Runner

Un exécuteur de tests (test runner) doit être fourni pour permettre l'exécution de Cucumber avec JUnit. Un tel exécuteur de tests (RunCucumberTests) est fourni ici dans le package cucumber sous le dossier source `src/test/kotlin`.

```

1    package cucumber
2
3    import io.cucumber.junit.platform.engine.Constants.GLUE_PROPERTY_NAME
4    import org.junit.platform.suite.api.ConfigurationParameter
5    import org.junit.platform.suite.api.IncludeEngines
6    import org.junit.platform.suite.api.SelectClasspathResource
7    import org.junit.platform.suite.api.Suite
8
9    @Suite
10   @IncludeEngines("cucumber")
11   @SelectClasspathResource("seg3x02.date")
12   @ConfigurationParameter(key = GLUE_PROPERTY_NAME, value = "cucumber" )
13   class RunCucumberTests {
14   }

```

Notez que le paramètre de l'annotation `@SelectClasspathResource` indique l'emplacement des fichiers de features dans le dossier des ressources (resources).

Intégration avec Spring Boot

La dépendance Cucumber pour Spring ("io.cucumber:cucumber-spring") doit être ajoutée au fichier de build Gradle. De plus, une configuration de tests Spring Boot annotée avec `@CucumberContextConfiguration` doit être fournie.

```

1    package seg3x02.auctionsystem.contracts
2
3    import io.cucumber.spring.CucumberContextConfiguration
4    import org.springframework.boot.test.context.SpringBootTest
5
6    @CucumberContextConfiguration
7    @SpringBootTest
8    class CucumberBootstrap {
9    }

```

Exécution

Avec un plugin Cucumber installé, votre IDE devrait fournir un mécanisme intégré pour exécuter les tests Cucumber.

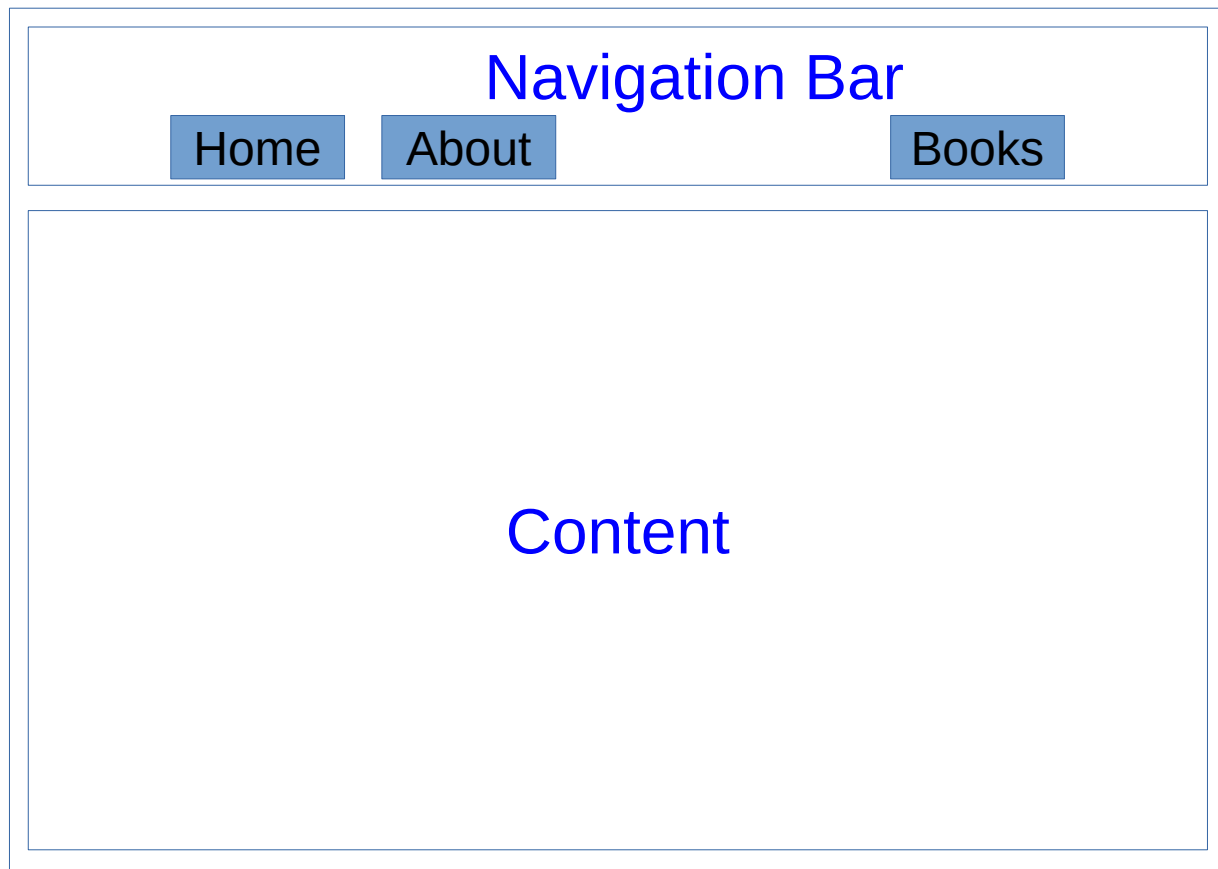
Les tests peuvent également être exécutés avec Gradle à l'aide de la commande `gradlew test`. La commande génère un rapport d'exécution de test accessible à `build/reports/test/index.html`.

II. Routage Angular

Nous introduisons le routage Angular dans cette partie du lab. Le code source est disponible dans le référentiel Github <https://github.com/stephanesome/angRouting.git>.

Librairie

Nous illustrons avec un prototype d'application Bookstore avec la mise en page suivante.



L'application présente une barre de navigation avec des boutons. Cliquer sur chacun de ces boutons affiche un contenu spécifique dans la zone Content.

Configuration de l'application

Créez une nouvelle application angular (`ng new book-store`).

Exécutez la commande `ng add @ng-bootstrap/ng-bootstrap` dans le dossier racine du projet pour ajouter Bootstrap au projet.

Routes

Ajoutez des routes au tableau `Routes` dans le module de routage `App Routing` comme suit.

```
1. export const routes: Routes = [  
2.   {path: 'home', component: Home},  
3.   {path: 'about', component: About},  
4.   {path: 'contact', component: Contact},  
5.   {path: 'books', component: Books},  
6.   {path: '', redirectTo: 'home', pathMatch: 'full'},  
7.   {path: '**', component: Home}];
```

Nous spécifions des routes vers

- le composant `Home` avec le chemin *home*,
- le composant `About` avec le chemin *about*,
- le composant `Contact` avec le chemin *contact* et
- le composant `Books` avec le chemin *books*.

De plus, une requête adressée à l'URL de base est redirigée vers *home* (ligne 6) ainsi que toutes les autres requêtes (ligne 7).

Générez les composants (`ng generate component home`, `ng generate component about`, `ng generate component contact`, `ng generate component books`). Ajoutez les importations des classes de composants générées à `src/app/app.routes.ts`.

Définissez les modèles HTML de `Home`, `About`, `Contact` en vous référant au code source sur Github. Assurez-vous de copier les images jpeg (`1books2-med.jpg` et `gobooks.jpg`) dans le répertoire `public`.

Routing Outlet

Modifiez le modèle HTML du composant `App` comme suit.

```
1. <div class="container">  
2.   <div class="nav-link">  
3.     <a [routerLink]="['/home']"> Home </a>  
4.     <a [routerLink]="['/about']"> About Us </a>  
5.     <a [routerLink]="['/contact']"> Contact Us </a>  
6.     <a [routerLink]="['/books']"> Books </a>  
7.   </div>  
8. </div>  
9.  
10. <div class="container-fluid" >  
11.   <div class="row">  
12.     <div class="col">  
13.         
14.     </div>  
15.     <div class="col-6">  
16.       <router-outlet></router-outlet>  
17.     </div>
```

```

18. <div class="col">
19.   
20. </div>
21. </div>
22. </div>

```

Le modèle définit la barre de navigation (lignes 3 à 9) à l'aide de l'attribut `routerLink` pour associer des routes à des balises d'ancrage de liens.

Le `router-outlet` à la ligne 16, spécifie où une route est rendue lorsqu'elle est sélectionnée.

Ajoutez des importations aux classes `RouterOutlet` et `RouterLink` à la classe de composant `App`.

```

1. import { Component } from '@angular/core';
2. import { RouterLink, RouterOutlet } from '@angular/router';
3.
4. @Component({
5.   selector: 'app-root',
6.   imports: [RouterOutlet, RouterLink],
7.   templateUrl: './app.html',
8.   styleUrls: ['./app.css']
9. })
10. export class App

```

Style CSS

Modifiez le fichier de style du composant `App` (`src/app/app.css`) comme suit.

```

1. a:link, a:visited {
2.   background-color: #0000b3;
3.   color: white;
4.   padding: 14px 25px;
5.   text-align: center;
6.   text-decoration: none;
7.   display: inline-block;
8. }
9.
10. a:hover, a:active {
11.   background-color: #e60000;
12. }

```

Composant Books

Supposons que nous souhaitons afficher un formulaire de recherche lorsqu'un utilisateur accède à l'URL `/books` en cliquant sur le bouton **Books**. Le formulaire invite l'utilisateur à entrer un identifiant de livre et affiche les détails de ce livre lorsqu'il est trouvé.

Nous devons naviguer vers une route spécifique au livre sélectionné. Cela ne peut pas être fait de manière statique car il y a potentiellement un grand nombre de livres et plus de livres peuvent être

ajoutés à la collection. Le routage d'angular offre la possibilité de créer des routes paramétrées pour ce scénario.

Route imbriquée

Nous devons d'abord spécifier la route des books comme une route imbriquée. Mettez à jour les routes dans `app.routes.ts` comme suit.

```
1. const booksRoutes: Routes = [  
2.   {path: 'id', component: Book}  
3. ];  
4.  
5. const routes: Routes = [  
6.   {path: 'home', component: Home},  
7.   {path: 'about', component: About},  
8.   {path: 'contact', component: Contact},  
9.   {path: 'books', component: Books,  
10.    children: booksRoutes  
11. },  
12. {path: '', redirectTo: 'home', pathMatch: 'full'},  
13. {path: '**', component: Home}  
14. ];
```

Ensuite, nous définissons les sous-routes (`booksRoutes`) comme des routes paramétrées (lignes 1-3). L'identifiant de l'itinéraire est un paramètre correspondant à un identifiant de livre ajouté au chemin de l'itinéraire parent (`/books`). Ces chemins rendent le composant `Book`.

Nous avons ajouté une propriété `children` à la route `books` (ligne 10). Cela indique une route imbriquée par rapport au composant `Books`.

Créez le composant `Book` (`ng generate component books/book`) et importez-le dans `app.routes.ts`.

Modèle HTML du composant Books

Modifiez le modèle HTML du composant `Books` comme suit.

```
1. <div>  
2.   <h2>Search our Book Collection</h2>  
3.   <div>  
4.     <label for="id">Book Id:</label>  
5.     <input type="text" #bookQuery id="id">  
6.     <button (click)="submit(bookQuery.value)">Search</button>  
7.   </div>  
8.   <div class="container">  
9.     <router-outlet></router-outlet>  
10.  </div>  
11. </div>
```

Le modèle HTML du composant `Books` présente une entrée pour entrer un identifiant pour le livre recherché, à la ligne 5. Le bouton `Search` à la ligne 6, soumet l'identifiant saisi par l'utilisateur. La ligne 9 spécifie une balise `router-outlet`, où la navigation vers le livre recherché est rendue.

Classe de composants Books

Modifiez la classe du composant Books comme suit.

```
1. import {Component, inject} from '@angular/core';
2. import { ActivatedRoute, Router, RouterOutlet } from '@angular/router';
3.
4. @Component({
5.   selector: 'app-books',
6.   templateUrl: './books.html',
7.   styleUrls: ['./books.css'],
8.   imports: [RouterOutlet]
9. })
10. export class Books {
11.   private router: Router = inject(Router);
12.   private route: ActivatedRoute = inject(ActivatedRoute);
13.
14.   submit(value: string): void {
15.     this.router.navigate(['./', value], {relativeTo: this.route}).then(() => {});
16.   }
17. }
```

Le routeur (router) et la route activée (activated route) sont injectés au composant à la ligne 11-12. La route activée est un observable qui renvoie les paramètres de la route courante. La navigation vers l'itinéraire sélectionné est effectuée par programmation en utilisant la fonction de navigation du routeur à la ligne 15 de la fonction submit.

Modèle BookEntity

Les informations relatives aux livres sont capturées comme une instance de la classe BookEntity. Générez la classe (ng generate class books/model/bookEntity) et modifiez src/app/books/model/book-entity.ts comme suit.

```
1. export class BookEntity {
2.   constructor(
3.     public id: number,
4.     public category: string,
5.     public title: string,
6.     public cost: number,
7.     public authors?: AuthorEntity[],
8.     public year?: number,
9.     public description?: string
10.  ) {}
11. }
12.
13. export class AuthorEntity {
14.   constructor(
15.     public firstName: string,
```

```
16.   public lastName: string
17.   }
18. }
```

Chaque livre est associé à une collection d'instances d'auteurs de la classe `AuthorEntity`.

Service Book

Un service est utilisé pour fournir un accès à la collection de livres. Générez le service (`ng generate service books/service/booksService`) et modifiez `src/app/books/service/books-service.ts` comme suit.

```
1. import { Injectable } from '@angular/core';
2. import { Book } from '../model/book';
3.
4. @Injectable({
5.   providedIn: 'root'
6. })
7. export class BooksService {
8.   private books = [
9.     new BookEntity(1001, 'Tech', 'Introduction to Angular', 50.25, [new AuthorEntity('Bob', 'T')], 2017),
10.    new BookEntity(1002, 'Tech', 'Angular Advanced Concepts', 125.95,
11.      [new AuthorEntity('Zorb', 'Tar')], 2019),
12.    new BookEntity(1003, 'Kids', 'A Fantastic Story', 12.25,
13.      [new AuthorEntity('Jane', 'C'), new AuthorEntity('Tala', 'Tolo')], 2009),
14.    new BookEntity(1004, 'Cook', 'The Best Shawarmas', 18.99, [new AuthorEntity('Chef', 'Z')], 1978),
15.    new BookEntity(1005, 'Tech', 'Angular Demystified', 210.50, [new AuthorEntity('Zorb', 'Tar')], 2020)
16.  ];
17.
18.   public getBook(id: string): BookEntity {
19.     return <BookEntity>this.books.find(book => book.id === Number.parseInt(id));
20.   }
21.
22.   public addBook(b: BookEntity): void {
23.     this.books.push(b);
24.   }
25. }
```

Le service code en dur une collection d'instances `BookEntity` et fournit une fonction qui renvoie un livre en fonction de son identifiant. Dans une application déployée, ce service interagirait avec un serveur pour obtenir les livres.

Composant Book

Modèle HTML

Modifiez le modèle HTML du composant Book (`src/books/book/book.html`) comme suit.

```

1.  @if (selectedBook) {
2.    <div>
3.      <h2>Here are details of the Book {{ selectedBook.id }}</h2>
4.      <div class="row">
5.        <div class="col-3">Category:</div>
6.        <div class="col-9">{{ selectedBook.category }}</div>
7.      </div>
8.      <div class="row">
9.        <div class="col-3">Title:</div>
10.       <div class="col-9">{{ selectedBook.title }}</div>
11.     </div>
12.     <div class="row">
13.       <div class="col-3">Cost:</div>
14.       <div class="col-9">{{ selectedBook.cost }}</div>
15.     </div>
16.     @if (selectedBook.authors) {
17.       <div class="row">
18.         <div class="col-3">Author:</div>
19.         <div class="col-9">
20.           <span [innerHTML]="selectedBook.authors | authornames"></span>
21.         </div>
22.       </div>
23.     }
24.     @if (selectedBook.year) {
25.       <div class="row">
26.         <div class="col-3">Year:</div>
27.         <div class="col-9">{{ selectedBook.year }}</div>
28.       </div>
29.     }
30.     @if (selectedBook.description) {
31.       <div class="row">
32.         <div class="col-3">Description:</div>
33.         <div class="col-9">{{ selectedBook.description }}</div>
34.       </div>
35.     }
36.   </div>
37. } @else {
38.   <div>
39.     <h2>Sorry can't find the requested book...</h2>
40.   </div>
41. }

```

Pipe nom d'Author

Le modèle HTML du composant Book utilise un pipe à la ligne 20 pour afficher correctement les noms des auteurs. Générez le pipe avec la commande `ng generate pipe pipes/authornames` et éditez `src/app/pipes/authornames.pipe.ts` comme suit.

```

1. import { Pipe, PipeTransform } from '@angular/core';
2. import { Author } from '../books/model/book';
3.
4. @Pipe({
5.   name: 'authornames',
6.   standalone: true
7. })
8. export class AuthornamesPipe implements PipeTransform {
9.
10.   transform(value: Author[] | undefined): string {
11.     if (value === undefined) return '';
12.     return value.map((author) => `${author.firstName}, ${author.lastName}`).join(' <b>and</b> ');
13.   }
14. }

```

Un pipe est créé en tant que classe décorée avec **@Pipe** qui implémente la fonction **transform** de l'interface **PipeTransform**. La fonction **transform** à la ligne 11, retourne une chaîne qui concatène chacun des **firstName** et **lastName** de l'auteur et joint avec *and*.

Class de Composant

Modifiez la classe de composants de **Book** (**src/books/book/book.ts**) comme suit.

```

1. import { Component, inject, OnInit } from '@angular/core';
2. import { ActivatedRoute } from '@angular/router';
3. import { Book } from '../model/book';
4. import { Books } from '../service/books.service';
5. import { AuthornamesPipe } from '../../pipes/authornames.pipe';
6. import { NgIf } from '@angular/common';
7.
8. @Component({
9.   selector: 'app-book',
10.  templateUrl: './book.html',
11.  styleUrls: ['./book.css'],
12.  imports: [NgIf, AuthornamesPipe]
13. })
14. export class Book implements OnInit {
15.   selectedBook!: Book;
16.   private route: ActivatedRoute = inject(ActivatedRoute);
17.   private booksService: BooksService = inject(BooksService);
18.
19.   ngOnInit(): void {
20.     this.route.params.subscribe(params => {
21.       const id = params['id'];
22.       this.selectedBook = this.booksService.getBook(id);
23.     });
24.   }
25. }

```

La route activée (**activated route**) et le service **booksService** sont injectés à la ligne 16-17. Le composant s'abonne en tant qu'observateur à la route activée (ligne 20) afin de récupérer le paramètre

d'itinéraire, l'identifiant du livre interrogé, puis le service de livres est utilisé pour rechercher un livre avec le identifiant (ligne 22).

Route authentifiée

Nous allons à présent ajouter une page d'administration pour permettre l'ajout de livres. Cette page doit cependant être accessible uniquement aux utilisateurs authentifiés. Par conséquent, nous utiliserons un route guard pour contrôler l'accès à la route de la page d'administration.

Routes

Modifiez le tableau de routes dans le module AppRoutingModule (src/app/app-routes.ts) comme suit.

```
1. export const routes: Routes = [  
2.   {path: 'home', component: Home},  
3.   {path: 'about', component: About},  
4.   {path: 'contact', component: Contact},  
5.   { path: 'login', component: Login },  
6.   {  
7.     path: 'admin',  
8.     component: Admin,  
9.     canActivate: [ loggedInGuard ]  
10.  },  
11.  {path: 'books', component: Books,  
12.    children: booksRoutes  
13.  },  
14.  {path: '', redirectTo: 'home', pathMatch: 'full'},  
15.  {path: '**', component: Home}  
16. ];
```

Nous avons ajouté un chemin *login* qui rend le composant Login (ligne 5) et un chemin *admin* qui rend Admin. La propriété *canActivate* du chemin *admin* est définie comme un tableau avec une garde. Générez le composant Login (ng generate component login), le composant Admin (ng generate component admin) et le LoggedIn Guard (ng generate guard logged-in) avec la méthode CanActivate. Assurez-vous d'importer les composants dans src/app/app-routes.ts.

Modifiez le modèle HTML du composant App comme suit.

```
1. <div class="container">  
2.   <p>Main Page</p>  
3.   <div class="nav-link">  
4.     <a [routerLink]="['/home']"> Home </a>  
5.     <a [routerLink]="['/about']"> About Us </a>  
6.     <a [routerLink]="['/contact']"> Contact Us </a>  
7.     <a [routerLink]="['/books']"> Books </a>  
8.     <a [routerLink]="['/login']"> Login </a>  
9.     <a [routerLink]="['/admin']"> Admin </a>
```

```

10. </div>
11. </div>
12.
13. <div class="container-fluid" >
14.   <div class="row">
15.     <div class="col">
16.       
17.     </div>
18.     <div class="col-6">
19.       <router-outlet></router-outlet>
20.     </div>
21.     <div class="col">
22.       
23.     </div>
24.   </div>
25. </div>

```

Nous avons ajouté des liens pour les chemins *admin* et *login*.

Service Authentication

Nous allons créer un service pour encapsuler la vérification de l'authentification des utilisateurs. Cette première version est basique mais avoir un service séparé aidera si/lorsque nous décidons d'utiliser une approche d'authentification plus sophistiquée car les changements seront contenus.

Générer un service Authentication (ng generate service authentication-service). Modifiez la classe AuthenticationService (src/app/authentication-service.ts) comme suit.

```

1. import {inject, Injectable} from '@angular/core';
2. import {Router} from '@angular/router';
3. import {noop} from 'rxjs';
4.
5. @Injectable({
6.   providedIn: 'root'
7. })
8. export class AuthenticationService {
9.   redirectUrl: string | null | undefined;
10.  private router: Router = inject(Router);
11.
12.  login(user: string, password: string): boolean {
13.    // hard coded for now
14.    if (user === 'admin' && password === 'password') {
15.      sessionStorage.setItem('username', user);
16.      if (this.redirectUrl) { this.router.navigate([this.redirectUrl]).then(noop); }
17.      this.redirectUrl = null;
18.      return true;
19.    }
20.    return false;
21.  }
22.
23.  logout(): any {

```

```

24.   sessionStorage.removeItem('username');
25. }
26.
27. getUser(): any {
28.   return sessionStorage.getItem('username');
29. }
30.
31. isLoggedIn(): boolean {
32.   return this.getUser() !== null;
33. }
34. }

```

Le service **Authentication** est utilisé pour la connexion, la déconnexion et pour vérifier qu'un utilisateur est authentifié. La fonction **login** aux lignes 12-21 (que nous utiliserons dans le composant **Login**) prend un id et un mot de passe (**password**) d'utilisateur et vérifie les informations d'identification de l'utilisateur. Des informations d'identification valides codées en dur sont utilisées ici. Dans une application en production, cela se ferait typiquement par une recherche d'utilisateurs stockés sur un serveur. Si les informations d'identification saisies sont valides, nous stockons l'utilisateur dans le **sessionStorage** du navigateur.

Nous voulons être amenés ici chaque fois qu'un utilisateur accède à un chemin protégé. Le service **Authentication** «se souvient» de l'URL dans la propriété **redirectUrl** et utilise le routeur (**router**) (injecté à la ligne 10) pour rediriger vers cette URL.

La fonction de déconnexion **logout** (lignes 23-25) efface simplement **sessionStorage** tandis que la fonction **isLoggedIn** (lignes 31-33) vérifie la présence d'un utilisateur authentifié dans **sessionStorage** à l'aide de la fonction **getUser** (lignes 27-29). **isLoggedIn** retourne **true** si un utilisateur authentifié est trouvé dans **sessionStorage**, **false** dans le cas contraire.

Garde Logged In

Nous pouvons maintenant compléter la Garde **LoggedIn**. Modifiez la fonction **loggedInGuard** (**src/app/logged-in.guard.ts**) comme suit.

```

1. import {CanActivateFn, Router} from '@angular/router';
2. import {AuthenticationService} from './authentication.service';
3. import {inject} from '@angular/core';
4.
5. export const loggedInGuard: CanActivateFn = (route, state) => {
6.   const authService = inject(AuthenticationService);
7.   const router= inject(Router);
8.
9.   // return this.authService.isLoggedIn();
10.  if (authService.isLoggedIn()) {return true; }
11.  authService.redirectUrl = state.url;
12.  router.navigate(['./login']).then(r => {});
13.  return false;
14. };

```

La fonction retourne `true` si une route protégée peut être activée, `false` dans le cas contraire. À la ligne 16, le service `Authentication`, injecté à la ligne 6, est utilisé pour vérifier que l'utilisateur de la session en cours est authentifié. Sinon, l'URL actuelle est stockée en tant que `redirectUrl` dans le service `Authentication` et le routeur, injecté à la ligne 7, redirige vers le chemin `login`.

Composant Login

Le composant `Login` obtient les informations d'identification d'un utilisateur et les vérifie à l'aide du service `Authentication`.

Modèle HTML du composant Login

Modifier le modèle HTML du composant `Login` (`src/app/login/login.html`) comme suit.

```
1. @if (message) {
2.   <div class="alert alert-danger" role="alert">
3.     {{ message }}
4.   </div>
5. }
6.
7. @if (isLoggedIn) {
8.   <div class="form-group">
9.     <p>Logged in as <b>{{ loggedUser }}</b></p>
10.    <button (click)="logout()" class="btn btn-danger">
11.      Log out
12.    </button>
13.  </div>
14. } @else {
15.   <div class="container">
16.     <div class="mb-2">
17.       <label for="username" class="form-label">UserName :</label>
18.       <input type="text" name="username" id="username" [(ngModel)]="username" class="form-control">
19.     </div>
20.     <div class="mb-2">
21.       <label for="password" class="form-label">Password :</label>
22.       <input type="password" name="password" id="password" [(ngModel)]="password" class="form-
23.         control">
24.     </div>
25.     <button (click)="checkLogin()" class="btn btn-success">
26.       Login
27.     </button>
28.   </div>
29. }
```

Le composant présente des champs de saisie pour le nom d'utilisateur et le mot de passe (lignes 8, 12) et les lie aux propriétés de la classe de composant. Le bouton `Login` à la ligne 14 déclenche alors la vérification des informations d'identification lorsque on clic dessus en appelant la fonction `checkLogin`.

Si un utilisateur est connecté, un autre bouton Log out s'affiche (ligne 20) et un clic dessus déclenche la déconnexion de l'utilisateur.

Classe de Composant Login

Modifiez la classe du composant Login (src/app/login/login.ts) comme suit.

```
1. import {Component, inject} from '@angular/core';
2. import {AuthenticationService} from '../authentication.service';
3. import { FormsModule } from '@angular/forms';
4. import { NgIf } from '@angular/common';
5.
6. @Component({
7.   selector: 'app-login',
8.   templateUrl: './login.html',
9.   styleUrls: ['./login.css'],
10.  imports: [NgIf, FormsModule]
11. })
12. export class Login {
13.   username = "";
14.   password = "";
15.   message!: string;
16.   private loginService: AuthenticationService = inject(AuthenticationService);
17.
18.   get isLoggedIn(): boolean {
19.     return this.loginService.isLoggedIn();
20.   }
21.
22.   get loggedUser(): string {
23.     return this.loginService.getUser();
24.   }
25.
26.   checkLogin(): boolean {
27.     this.message = "";
28.     if (!this.loginService.login(this.username, this.password)) {
29.       this.message = 'Invalid Login';
30.       setTimeout(() => {
31.         this.message = "";
32.       }, 2500);
33.       return false;
34.     }
35.     return true;
36.   }
37.
38.   logout(): boolean {
39.     this.loginService.logout();
40.     return true;
41.   }
42. }
```

Le service `Authentication` est injecté à la ligne 16. Dans la fonction `checkLogin`, nous utilisons le service `Authentication` pour vérifier les informations d'identification saisies (ligne 28). Si la connexion de l'utilisateur est refusée, nous définissons un message et utilisons un minuteur pour qu'il s'affiche pendant une durée donnée (lignes 27-32).

Composant Admin

Nous offrons maintenant la possibilité d'ajouter plus de livres à la librairie dans le composant `Admin`. Nous utilisons également des formulaires réactifs et dynamiques et la validation de formulaire.

Classe de Composant Admin

Modifiez la classe du composant `Admin` (`src/app/admin/admin.ts`) comme suit.

```
1. import {Component, inject} from '@angular/core';
2. import { AbstractControl, FormArray, FormBuilder, FormControl, Validators, FormsModule,
   ReactiveFormsModule } from '@angular/forms';
3. import {Author, Book} from '../books/model/book';
4. import {BooksService} from '../books/service/books.service';
5. import { NgFor } from '@angular/common';
6.
7. function categoryValidator(control: FormControl<string>): { [s: string]: boolean } | null {
8.   const validCategories = ['Kids', 'Tech', 'Cook'];
9.   if (!validCategories.includes(control.value)) {
10.    return {invalidCategory: true};
11.   }
12.   return null;
13. }
14.
15. @Component({
16.   selector: 'app-admin',
17.   templateUrl: './admin.html',
18.   styleUrls: ['./admin.css'],
19.   imports: [FormsModule, ReactiveFormsModule, NgFor]
20. })
21. export class Admin {
22.   private builder: FormBuilder = inject(FormBuilder);
23.   private booksService: BooksService = inject(BooksService);
24.   bookForm = this.builder.group({
25.     id: ['', [Validators.required, Validators.pattern('[1-9]\d{3}')]],
26.     category: ['', [Validators.required, categoryValidator]],
27.     title: ['', Validators.required],
28.     cost: ['', [Validators.required, Validators.pattern('\d+(\.\d{1,2})?')]],
29.     authors: this.builder.array([]),
30.     year: [''],
31.     description: ['']
32.   });
```

```

33.
34. get id(): AbstractControl<string> {return <AbstractControl>this.bookForm.get('id'); }
35. get category(): AbstractControl<string> {return <AbstractControl>this.bookForm.get('category'); }
36. get title(): AbstractControl<string> {return <AbstractControl>this.bookForm.get('title'); }
37. get cost(): AbstractControl<string> {return <AbstractControl>this.bookForm.get('cost'); }
38. get authors(): FormArray {
39.   return this.bookForm.get('authors') as FormArray;
40. }
41.
42. onSubmit(): void {
43.   const book = new Book(Number(this.bookForm.value.id),
44.     <string>this.bookForm.value.category,
45.     <string>this.bookForm.value.title,
46.     Number(this.bookForm.value.cost),
47.     <Author[]>this.bookForm.value.authors,
48.     Number(this.bookForm.value.year),
49.     <string>this.bookForm.value.description);
50.   this.booksService.addBook(book);
51.   this.bookForm.reset();
52.   this.authors.clear();
53. }
54.
55. addAuthor(): void {
56.   this.authors.push(
57.     this.builder.group({
58.       firstName: [""],
59.       lastName: [""]
60.     })
61.   );
62. }
63.
64. removeAuthor(i: number): void {
65.   this.authors.removeAt(i);
66. }
67. }

```

Le composant Admin utilise un formulaire réactif (Reactive Form). Assurez-vous d'importer **FormsModule** et **ReactiveFormsModule** dans **AppModule**. Ces modules doivent être spécifiés dans le tableau des imports du décorateur **@NgModule**.

Le formulaire **bookForm**, est défini aux lignes 24-32 à l'aide du service **FormBuilder** injecté à la ligne 22. Le builder crée un groupe de formulaire (*form group*) dans lequel les champs de formulaires (*form controls*) sont déclarés en tant que paires valeurs propriété. Chaque champ de formulaire peut être associé à une liste de validateurs. Par exemple, les validateurs de l'identifiant sont **Validators.required** et **Validators.pattern("[1-9]\\d{3}')**. Le premier validateur échoue si une valeur n'est pas fournie pour le contrôle tandis que le second assure que la valeur fournie satisfait une expression régulière. **categoryValidator** utilisé pour *category* à la ligne 26, est un validateur personnalisé défini aux lignes 7-13. Il vérifie que la valeur fournie se trouve dans une liste de catégories valides.

Le contrôle *authors* est un **FormArray** qui permet la création dynamique de contrôles de formulaires. Nous utilisons ceci pour qu'un nombre variable de noms d'auteurs puisse être capturé. La fonction **addAuthor** aux lignes 55-62, ajoute un auteur tandis que la fonction **removeAuthor** aux lignes 64-66 supprime un auteur du *form array*.

Nous spécifions des getters aux lignes 34-40 pour faire référence aux contrôles de formulaire plus facilement dans le modèle HTML.

La fonction **onSubmit** (lignes 42-53) est le gestionnaire de soumission du formulaire. Il crée une instance de la classe du modèle **Book** à partir des valeurs des contrôles du formulaire et appelle le service **Books** (injecté à la ligne 23), pour ajouter la nouvelle instance à la collection de livres.

Ajoutez la fonction **addBook** à la classe **BookService** (`src/app/books/service/books.ts`).

```
1. public addBook(b: BookEntity): void {  
2.     this.books.push(b);  
3. }
```

La fonction ajoute un livre à la collection de livres.

Modèle HTML du composant Admin

Modifier le modèle HTML du composant Admin (`src/app/admin/admin.html`) comme suit.

```
1. <div class="container">  
2.   <h1>BookEntity Form</h1>  
3.   <button (click)="addAuthor()" class="btn btn-primary">Add Author</button>  
4.   <form [formGroup]="bookForm" (ngSubmit)="onSubmit()">  
5.  
6.     <div class="mb-3">  
7.       <label for="id">Id:</label>  
8.       <input type="text" class="form-control" id="id" formControlName="id" required>  
9.       @if (id.dirty && !id.valid) {  
10.        <div class="alert alert-danger">  
11.          Id must be 4 digits long starting with a number different from 0.  
12.        </div>  
13.      }  
14.    </div>  
15.  
16.    <div class="mb-3">  
17.      <label for="category">Category:</label>  
18.      <input type="text" class="form-control" id="category" formControlName="category">  
19.      @if (category.dirty && !category.valid) {  
20.        <div class="alert alert-danger">  
21.          Category is required to be <b>Kids</b>, <b>Tech</b> or <b>Cook</b>  
22.        </div>  
23.      }  
24.    </div>  
25.  
26.    <div class="mb-3">
```

```

27. <label for="title">Title:</label>
28. <input type="text" class="form-control" id="title" required formControlName="title">
29. @if (title.dirty && !title.valid) {
30.   <div class="alert alert-danger">
31.     Title is required.
32.   </div>
33. }
34. </div>
35.
36. <div class="mb-3">
37.   <label for="cost">Cost:</label>
38.   <input type="text" class="form-control" id="cost" required pattern="\d+(\.\d{1,2})?"
formControlName="cost">
39.   @if (cost.dirty && !cost.valid) {
40.     <div class="alert alert-danger">
41.       Cost should be a number with two optional decimals
42.     </div>
43.   }
44. </div>
45.
46. <div formArrayName="authors">
47.   @for (author of authors.controls; track $index) {
48.     <ng-container [formGroupName]="$index">
49.       <div class="row g-lg-5 align-items-center">
50.         <div class="col-auto">
51.           <label for="authName-{{$index}}" class="col-form-label">Author First Name:</label>
52.         </div>
53.         <div class="col-auto">
54.           <input id="authName-{{$index}}" class="form-control" formControlName="firstName"
type="text">
55.         </div>
56.       </div>
57.       <div class="row g-lg-5 align-items-center">
58.         <div class="col-auto">
59.           <label for="authLName-{{$index}}" class="col-form-label">Author Last Name:</label>
60.         </div>
61.         <div class="col-auto">
62.           <input id="authLName-{{$index}}" class="form-control" formControlName="lastName"
type="text">
63.         </div>
64.         <div class="col-auto">
65.           <button class="btn btn-dark form-control" type="button"
(click)="removeAuthor($index)">X</button>
66.         </div>
67.       </div>
68.     </ng-container>
69.   }
70. </div>
71.

```

```

72. <div class="mb-3">
73.   <label for="year">Year:</label>
74.   <input type="text" class="form-control" id="year" formControlName="year">
75. </div>
76.
77. <div class="mb-3">
78.   <label for="description">Description:</label>
79.   <textarea cols="40" class="form-control" id="description"
      formControlName="description"></textarea>
80. </div>
81.
82. <button type="submit" class="btn btn-success" [disabled]="bookForm.invalid">Submit</button>
83. </form>
84. </div>

```

Le modèle HTML présente le formulaire défini dans la classe du composant. Les erreurs de validation sont affichées dans un *div* qui est masqué lorsqu'un contrôle de formulaire est valide (*valid*) ou vierge (*pristine* - sa valeur n'a pas encore été modifiée).

III. Exercise

L'exercice suivant est le livrable pour le laboratoire. Complétez et enregistrez le code dans Github Classroom avant la date limite. Seul le travail de ce exercice sera évalué.

Implémentez une application Angular à l'aide d'un formulaire réactif. L'application présentera un formulaire dans lequel les utilisateurs peuvent soumettre les informations suivantes:

- Prénom
- Nom de famille
- Numéro de téléphone
- Email

Une validation doit être effectuée pour vérifier que les valeurs saisies satisfont aux contraintes suivantes.

- Le prénom et le nom sont obligatoires.
- Le numéro de téléphone, lorsqu'il est fourni, doit comporter 10 chiffres avec les premier et quatrième chiffres différents de 0.
- L'e-mail doit être conforme à un modèle d'e-mail valide (utilisez `Angular Validators.email ()`).

L'application présentera une nouvelle page et y affichera les données d'utilisateur saisies dans une table lorsque des informations valides seront soumises. Des messages d'erreurs appropriés seront affichés au cas où les entrées sont invalides.